Assignment: Multi-Agent Trip Planner using LangGraph

With Memory + Orchestrator Agent + PDF Generator

This project is a **Production-Grade Agentic AI System** built using **LangGraph**, where multiple specialized agents collaborate under the control of an **Orchestrator Agent (Supervisor Agent)** to generate a complete personalized trip plan.

The system includes:

- Multi-Agent Workflow
- API Integrations
- Memory System
- Supervisor / Orchestrator Agent
- Final PDF Downloadable Report

This is one of the best real-world portfolio projects for GenAI Engineer roles.

Problem Statement

Build an AI-powered **Trip Planning Assistant** that helps users plan complete trips from start to finish.

The system should take inputs like:

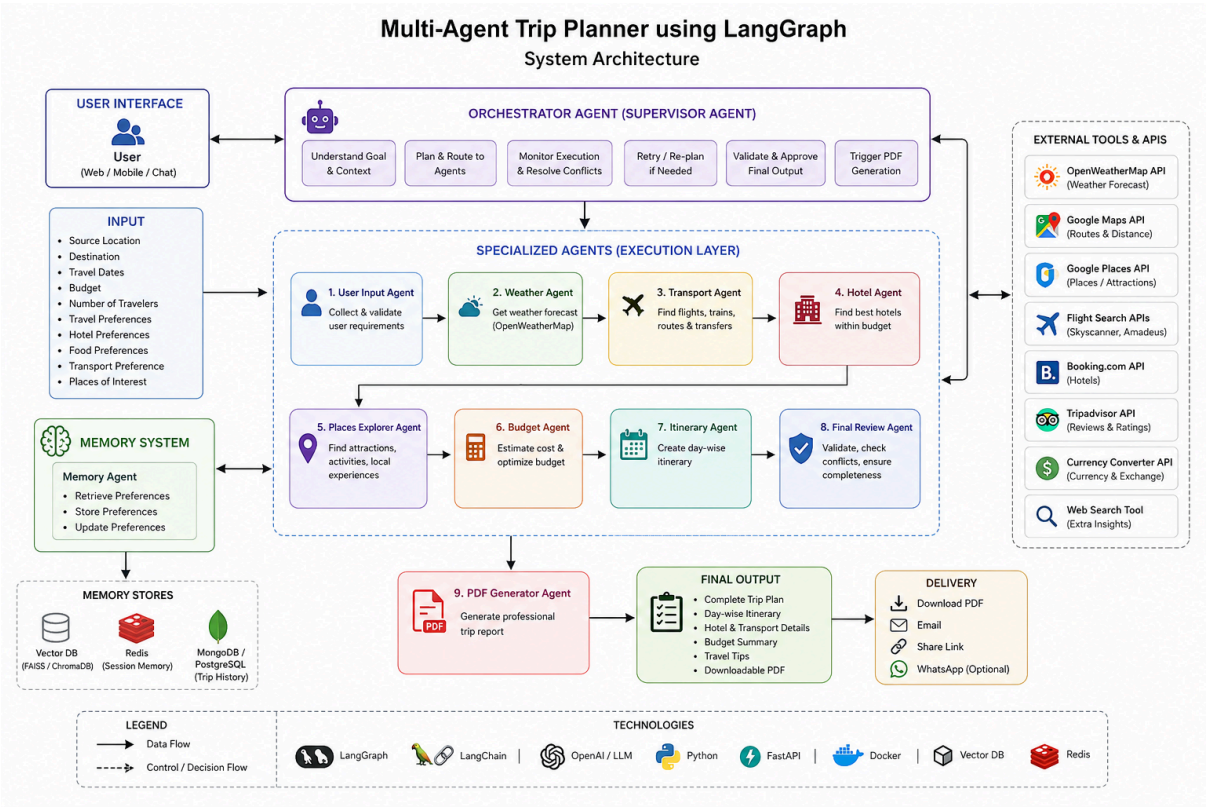
- Source location
- Destination
- Travel dates
- Budget
- Number of travelers
- Travel type (solo/family/couple/business)
- Hotel preferences
- Food preferences
- Transport preferences
- Places of interest
- Luxury vs Budget preference

and generate:

- Best routes
- Travel recommendations
- Flight/train/car suggestions
- Hotel recommendations
- Weather updates
- Tourist attractions
- Day-wise itinerary
- Budget optimization
- Local food suggestions
- Emergency travel tips
- Final downloadable PDF report

The system must use **LangGraph Multi-Agent Architecture** with one **Orchestrator Agent** controlling all specialized agents.

System Architecture



Most Important Agent

Orchestrator Agent (Supervisor Agent)

Responsibility

This is the brain of the entire system.

It decides:

- which agent should run
- when an agent should run
- whether an agent needs retry
- whether enough information is available
- whether the final answer is valid
- whether memory should be updated
- when PDF generation should happen

This is the **Manager Agent** of the system.

Without this, agents are just independent workers.

With this, the system becomes true **Agentic AI**.

Orchestrator Agent Tasks

Step 1 — Understand User Goal

Analyze:

- what the user wants

- missing information
- urgency
- complexity

Step 2 — Route to Correct Agents

Decide:

- should Weather Agent run first?
- should Hotel Agent run before Budget Agent?
- should Memory be checked first?
- should Final Review happen again?

Step 3 — Resolve Conflicts

Example:

Hotel exceeds budget

↓

Ask Budget Agent for optimization

OR

Ask Hotel Agent for better options

Step 4 — Retry Failed Tasks

Example:

Flight API fails

↓

Retry using backup API

Step 5 — Final Approval

Before output:

- verify completeness
- verify correctness
- approve final itinerary

Step 6 — Trigger PDF Generator

Only after approval:

Generate final downloadable report

Full Agent List

1. User Input Agent

Collect user requirements

2. Memory Agent

Retrieve and update past preferences

3. Weather Agent

Fetch weather forecast using OpenWeatherMap

4. Transport Agent

Flights / train / routes / transfers

5. Hotel Agent

Find hotels within budget

---

6. Places Explorer Agent

Tourist attractions + local experiences

---

7. Budget Agent

Cost optimization and estimation

---

8. Itinerary Agent

Generate day-wise trip planning

---

9. Final Review Agent

Validation + conflict detection

---

10. PDF Generator Agent

Generate professional travel report PDF

---

11. Orchestrator Agent

Controls everything

This is the most important agent.

Tools Used

Tool	Purpose
OpenWeatherMap API	Weather
Google Maps API	Routes
Google Places API	Attractions
Flight Search APIs	Flights

Booking.com API	Hotels
Tripadvisor API	Reviews
Vector DB (FAISS / ChromaDB)	Memory
Redis	Session memory
PostgreSQL / MongoDB	Trip history
Python Calculator	Budget
ReportLab / FPDF	PDF generation
Web Search Tool	Extra travel insights

Final LangGraph Workflow

START

→ Orchestrator Agent

→ User Input Agent

→ Memory Retrieval Agent

→ Orchestrator Decision

→ Parallel Execution:

→ Weather Agent

→ Hotel Agent

→ Transport Agent

→ Places Agent

→ Budget Agent

→ Itinerary Agent

→ Final Review Agent

→ Orchestrator Validation

IF approved:

→ Memory Update Agent

→ PDF Generator Agent

ELSE:

→ Retry required agents

→ FINAL OUTPUT

→ END

State Schema Example

{
"user_profile": {},
"trip_preferences": {},
"weather_data": {},
"hotel_data": {},
"transport_data": {},
"places_data": {},
"budget_summary": {},
"itinerary": {},
"review_status": {},
"pdf_status": {},
"orchestrator_decision": {}
}

Example Orchestrator Logic

If budget < hotel estimate

→ Re-run Hotel Agent

If weather = heavy rain

→ Re-run Places Agent

If flight unavailable

→ Re-run Transport Agent

If itinerary conflict exists

→ Re-run Itinerary Agent

Final PDF Sections

Cover Page

Trip Summary

Section 1

Flights + Transfers

Section 2

Hotel Details

Section 3

Day-wise Itinerary

Section 4

Budget Report

Section 5

Packing Checklist

Section 6

Emergency Contacts

Advanced Bonus Features

Can Be Added

Voice-based Trip Planning

WhatsApp PDF Delivery

Live Flight Tracking

Dynamic Replanning

Weather Alerts

Expense Tracker

QR-based Booking Links

Visa Guidance

Group Trip Collaboration

Final Deliverables

Students Must Submit:

- Architecture Diagram
- LangGraph Workflow
- Agent Design
- Orchestrator Logic
- Tool Design

- Memory System
- PDF Generator Module
- Prompt Design
- State Schema
- GitHub Repository
- Working Demo
- Final PDF Output

Sample User Query

Plan a 5-day Goa trip from Bangalore for a couple

Budget: ₹30,000

Need:

- beach resort

- nightlife

- sightseeing

- seafood

- flight preferred

Expected Final Result

A professional AI Travel Consultant system that:

- remembers users
- plans smartly
- coordinates multiple agents
- resolves conflicts
- generates beautiful reports